

Tower Defense Game with Procedurally Generated Content and Rogue-like Elements

Vilém Gutvald (vildagut@seznam.cz), 2024

INTRODUCTION

Tower defense is a subgenre of strategy games, where the player defends against waves of attackers by building defensive towers. We believe that adding some mechanics from **rogue-like** games could make for a great game. In this thesis, we designed and implemented such a game in Unity using C#. In it, the players go through randomized procedurally generated levels with tower defense gameplay. They collect new towers and other blueprints from randomized offers as they go, trying to get as far as possible.

THESIS GOALS

- The main goals of the thesis were:
- **Design** the game's mechanics and features.
 - **Implement** a playable demo version of the game.
 - Conduct a **playtest** to gather feedback.

GAME DESIGN

The first goal of our thesis was to design the game. To do this, **we analyzed other games**, namely *Plants vs Zombies* and *Slay the Spire*. By identifying what makes them enjoyable, **we established design goals** specific to our game. The game aims to be strategically interesting on multiple levels, it should be challenging, and it should encourage the players to explore new strategies. With these goals in mind, we designed the game's systems and mechanics, and created a sort of design document.

DEMO GAME FEATURES

Based on our design, we implemented a demo version of the game to test its core mechanics. A screenshot can be seen in the figure below:



In the screenshot above, we can see the game world:

1. **procedurally generated 3D terrain**, which plays an important role in the gameplay.
 2. **Attackers** coming along attacker paths, some with **special abilities**.
 3. Their target — **the Hub**. When an attacker reaches it, the player loses some **hull**.
- The player has placed several buildings in the world:
4. Surface drills **collecting materials**, which the player needs to build more buildings.
 5. Various **towers shooting at attackers**, each with a unique behavior and strengths.
- The graphical user interface communicates to the player important information:
6. The **current wave** and the number of remaining waves.
 7. Player's **hull** (when it reaches zero, the game is over).
 8. **Preview** of the composition of **next waves**, so the player can **plan ahead**.
 9. The player's current amount of **material** (orange) and **energy** (blue).
 10. **Blueprint menu** with the blueprints the player can currently use. The ones shown here are **abilities that deal damage** to attackers within some radius when used.
 11. Current **level number** and run **seed** which completely determines this run's levels.

We made a special effort to ensure everything was as clear as possible for the player. For example, in the figure below, we can see:

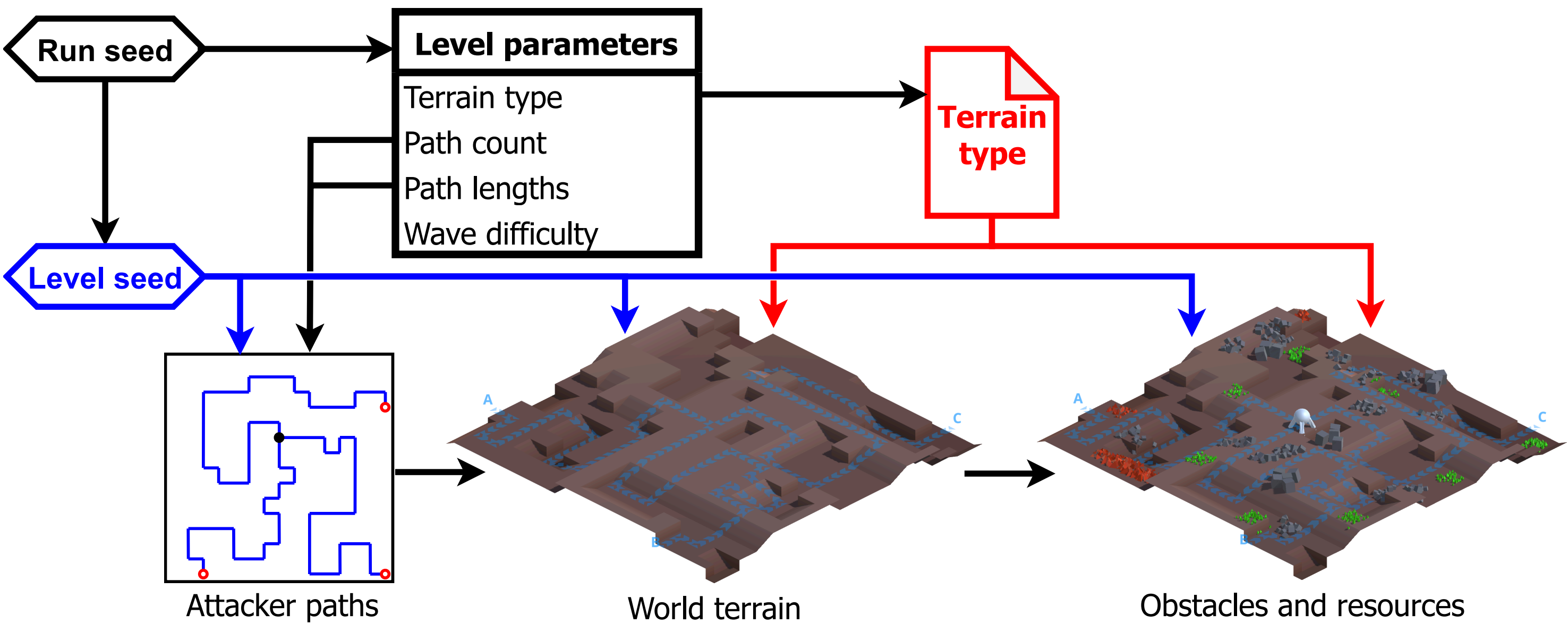
12. A **Radar** building, which increases the range of adjacent towers.
13. And a **Gatling Gun** tower. It is currently **selected**, as denoted by the **blue highlight**. We can also see its **range** in **green**, visualising the tower's **line of sight**.

On the right, we show the in-game **info panel** that shows information about the selected Gatling Gun. Its range statistic is **green** because it is **increased by the radar**, and some of its statistics are **red** because it needs a while to **spin up**, as per its description.



PROCEDURALLY GENERATING LEVELS

The levels and blueprint rewards in each run are randomized. However, this randomness is **fully determined by the run seed**. The image below shows a simplified overview of the level generation. Based on the run seed, we generate the level seed and the level parameters. We have set many requirements for the generated worlds, and to fulfill them, we had to come up with sophisticated algorithms.



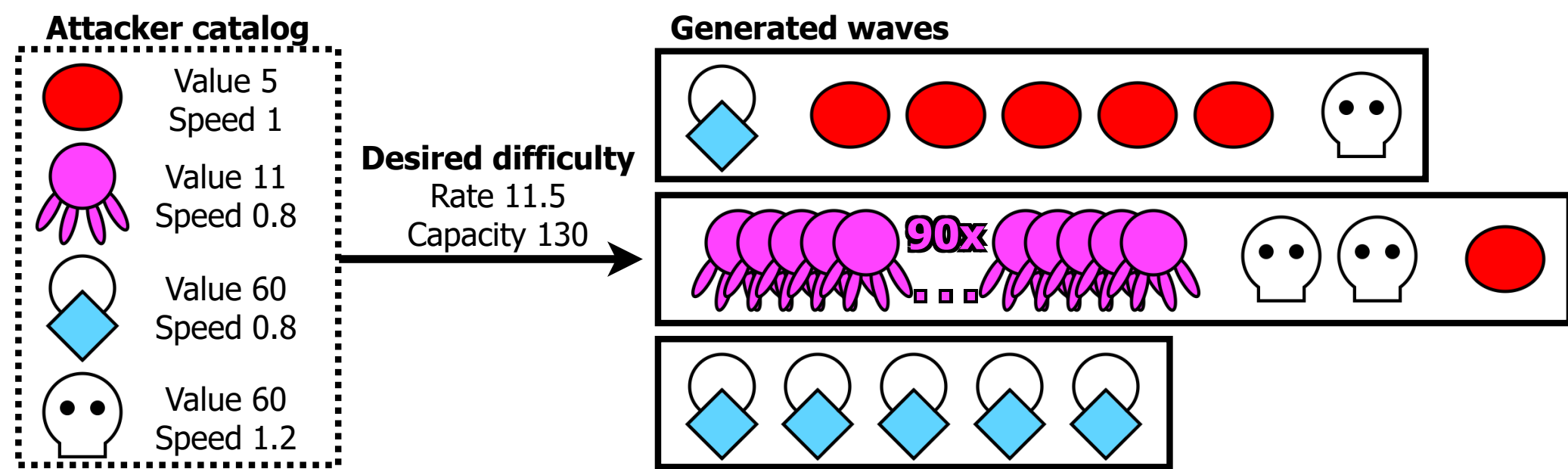
First, we generate the **attacker paths** based on level parameters such as the path lengths. This is done in several steps, for which we developed custom algorithms. For example, we adapted **simulated annealing** to use it to optimize the path shapes. This lets us create complex path networks that feel **unique but fair**.

Then the **3D terrain** is generated. To allow different levels to look and feel more different, the terrain components are specified using a custom **terrain type** file. To generate the terrain, we use our custom implementation of **wave function collapse**. This lets us generate randomized terrain from the **components we specify** that also respects many requirements, for example that paths cannot go through a cliff.

Finally, we distribute **obstacles and resources** on the generated world, which are also customizable in the terrain type file. To make them **look natural**, we procedurally generate their appearance by randomly **scattering models** on the world surface.

PROCEDURALLY GENERATING ATTACKER WAVES

In each level, the player has to defend against several waves of attackers. Each wave is procedurally generated from a catalog of attackers, such that it has the desired difficulty. Since there is no algorithm which satisfies our requirements, we use a **custom algorithm**. To create it, we first had to develop a way to **quantify wave difficulty** that is accurate enough to generate fair waves. Below is an example of different waves generated with the same given desired difficulty and attacker catalog:



IN-GAME TUTORIAL

The game includes an **interactive tutorial** that guides players and explains the main mechanics. This tutorial proved **very effective**, allowing players to play and learn more without external help. One step of the tutorial is shown in the image below.

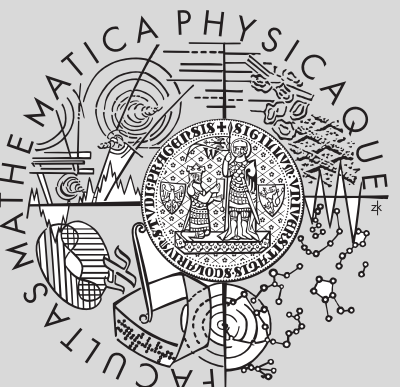


PLAYTEST RESULTS

Finally, we conducted a playtest and gathered feedback from **11 players**. During the playtesting, we fixed few bugs the players found, and we tweaked the game balance to make it more fun. The feedback showed that the game's systems were designed well, and overall, the **players enjoyed the game**. It has **a lot of strategic depth** even as a demo version. However, the game also has several issues. Some are addressed by the features we planned for later, others will require some systems to be redesigned.

CONCLUSION

In conclusion, **we achieved our goals** and developed an **engaging game** with great potential. The algorithms we developed effectively address the problems they were designed to solve. However, there is still more work to be done, including implementing planned features, fixing issues, and adding more content.



CHARLES UNIVERSITY
Faculty of mathematics
and physics

Supervisor: Mgr. Pavel Ježek, Ph.D.
Department of Distributed and Dependable Systems



Game (Windows)



Thesis